Université Cheikh Anta Diop

Ecole Supérieure Polytechnique
Département Génie Informatique
Année Universitaire 2025-2026
Licence en Génie Logiciel et Système d'Information
Analyse des algorithmes

Rapport du projet de Fin de semestre

Groupe 1 : Daba SENE, Aminata Ndiaye, Arame Yvonne Diop, Khadim Ndiaye

1. Introduction
2. Description du module
3. Fonctions utilitaires
    3.1 generer_table()
    3.2 aplatir()
    3.3 reconstruire()
4. Algorithmes de tri
    4.1 Tri à bulles
    4.2 Tri par sélection
    4.3 Tri par insertion
    4.4 Tri rapide (Quick Sort)
    4.5 Tri par fusion (Merge Sort)
5. Algorithmes de recherche
    5.1 Recherche linéaire
    5.2 Recherche binaire
    5.3 Recherche par dichotomie
6. Tests et validation
7. Difficultés rencontrées
Conclusion

1. Introduction

Dans le cadre de ce projet, nous avons développé un module Python permettant d'effectuer des opérations de tri et de recherche sur des tables à deux dimensions.

Une table à deux dimensions est une structure de données organisée en lignes et en colonnes, similaire à un tableau Excel. En Python, elle est représentée par une liste de listes. Par exemple, un tableau de 4 lignes et 5 colonnes contenant des valeurs entières aléatoires se présente ainsi :

[[34, 12, 87, 5, 23],

[67, 8, 91, 45, 72],

[3, 56, 19, 88, 41],

[60, 27, 74, 13, 95]]

L'objectif est de fournir, sous forme de module importable, cinq algorithmes de tri : le tri à bulles, le tri par sélection, le tri par insertion, le tri rapide et le tri par fusion ; ainsi que trois algorithmes de recherche : la recherche linéaire, la recherche binaire et la recherche par dichotomie.

Tous les algorithmes de tri reposent sur une même logique en trois étapes : aplatir le tableau 2D en une liste à une dimension, trier cette liste, puis reconstruire le tableau avec ses dimensions d'origine.

```
========================================
  Tableau original (aléatoire)
========================================
    10    86    36    59    72
    17    41    26    51    84
    77    66     2    65    56
     7    41    62    74     8
========================================
```

2. Description du module

Le projet est composé de deux fichiers Python. Le fichier tableau2d.py contient toutes les fonctions du module : la génération du tableau, les algorithmes de tri et les algorithmes de recherche. Le fichier test_tableau2d.py permet de tester et de vérifier que chaque fonction fonctionne correctement.

Le point central du module est la façon dont on gère le tableau 2D pour le trier. Les algorithmes de tri classiques travaillent sur des listes simples à une dimension. Pour les appliquer sur un tableau 2D, on passe par trois étapes :

a. On aplatit le tableau 2D en une liste 1D
b. On trie cette liste
c. On reconstruit le tableau 2D avec les dimensions d'origine

Par exemple, le tableau suivant :
[[3, 1],
[4, 2]]
devient la liste : [3, 1, 4, 2]
Après tri : [1, 2, 3, 4]
Et on reconstruit :
[[1, 2],
[3, 4]]

```
Generation de la table 2D aleatoire...
Table originale :
[2, 29, 64, 54]
[94, 23, 44, 40]
[90, 40, 86, 94]
[53, 85, 46, 78]
Liste aplatie : [2, 29, 64, 54, 94, 23, 44, 40, 90, 40, 86, 94, 53, 85, 46, 78]
```

```
==========================
   TESTS DES TRIS
==========================

Test tri a bulle :
[2, 23, 29, 40]
[40, 44, 46, 53]
[54, 64, 78, 85]
[86, 90, 94, 94]
```

3. Fonctions utilitaires

Ces fonctions sont la base du module. Elles sont utilisées par tous les algorithmes de tri et de recherche.

3.1 generer_table()

Cette fonction crée un tableau 2D rempli de valeurs entières aléatoires. On lui passe le nombre de lignes, le nombre de colonnes, et les valeurs minimale et maximale souhaitées.

```
def generer_table(lignes=4, colonnes=4, min_val=0, max_val=100):
    return [[random.randint(min_val, max_val) for _ in range(colonnes)] for _ in range(lignes)]
```

La fonction utilise **random.randint(min_val, max_val)** qui génère un entier aléatoire entre les deux bornes. On répète ça pour chaque colonne, et pour chaque ligne, ce qui donne un tableau complet.

3.2 aplatir()

```
def aplatir(table):
    return [element for ligne in table for element in ligne]
```

Cette fonction parcourt chaque ligne du tableau, et pour chaque ligne elle parcourt chaque élément. Le résultat est une liste simple à une dimension. C'est une étape obligatoire avant d'appliquer un algorithme de tri.

3.3 reconstruire()

Après le tri, on redécoupe la liste 1D en lignes de taille **nb_colonnes**. La formule **liste_1d[i:i + nb_colonnes]** prend une tranche de la liste à chaque itération. On obtient ainsi le tableau 2D reconstitué avec les bonnes dimensions.

```
def reconstruire(liste_1d, nb_colonnes):
    return [liste_1d[i:i + nb_colonnes] for i in range(0, len(liste_1d), nb_colonnes)]
```

4. Algorithmes de tri

Tous les algorithmes de tri suivent la même logique : on aplatit le tableau 2D en liste 1D, on trie, puis on reconstruit le tableau. Ce qui change d'un algorithme à l'autre, c'est la façon dont on trie la liste.

4.1 Tri à bulles

```
def tri_bulle(table):
    nb_colonnes = len(table[0])
    lst = aplatir(table)
    n = len(lst)

    for i in range(n):
        echange = False
        for j in range(0, n - i - 1):
            if lst[j] > lst[j + 1]:
                lst[j], lst[j + 1] = lst[j + 1], lst[j]
                echange = True
        if not echange:
            break

    return reconstruire(lst, nb_colonnes)
```

Le tri à bulles compare chaque élément avec son voisin de droite. Si l'élément de gauche est plus grand, on les échange. On répète ce processus jusqu'à ce qu'aucun échange ne soit fait, ce qui veut dire que la liste est triée. Les grandes valeurs remontent progressivement vers la fin, comme des bulles.

On remarque que plus la liste est grande, plus ce tri devient lent, parce qu'on fait beaucoup de comparaisons inutiles.

Complexité : pire cas $O(n^2)$, meilleur cas $O(n)$ quand la liste est déjà triée.

```
Test tri a bulle :
[2, 23, 29, 40]
[40, 44, 46, 53]
[54, 64, 78, 85]
[86, 90, 94, 94]
```

4.2 Tri par sélection

```python
def tri_selection(table):
    nb_colonnes = len(table[0])
    lst = aplatir(table)
    n = len(lst)

    for i in range(n):
        idx_min = i
        for j in range(i + 1, n):
            if lst[j] < lst[idx_min]:
                idx_min = j
        lst[i], lst[idx_min] = lst[idx_min], lst[i]

    return reconstruire(lst, nb_colonnes)
```

Le tri par sélection divise mentalement la liste en deux parties : la partie gauche déjà triée, et la partie droite pas encore triée. A chaque étape, on cherche le plus petit élément dans la partie non triée, et on le place à la fin de la partie triée en l'échangeant avec le premier élément non trié.
Contrairement au tri à bulles, on fait exactement un seul échange par tour de boucle. Ca le rend un peu plus propre dans son fonctionnement, mais pas forcément plus rapide.
Complexité : $O(n^2)$ dans tous les cas, même si la liste est déjà triée, parce qu'on cherche toujours le minimum.

```
Test tri par selection :
[2, 23, 29, 40]
[40, 44, 46, 53]
[54, 64, 78, 85]
[86, 90, 94, 94]
```

4.3 Tri par insertion

```python
def tri_insertion(table):
    nb_colonnes = len(table[0])
    lst = aplatir(table)
    n = len(lst)

    for i in range(1, n):
        cle = lst[i]
        j = i - 1

        while j >= 0 and lst[j] > cle:
            lst[j + 1] = lst[j]
            j -= 1

        lst[j + 1] = cle

    return reconstruire(lst, nb_colonnes)
```

Le tri par insertion fonctionne comme quand on trie des cartes dans sa main. On prend une carte, et on la glisse à sa bonne place parmi les cartes déjà triées. On répète ça jusqu'à la dernière carte.
Dans le code, on appelle **cle** l'élément qu'on est en train d'insérer. On remonte dans la partie déjà triée tant qu'on trouve des éléments plus grands que cette clé, en les décalant d'une position vers la droite. Quand on trouve la bonne place, on insère la clé.
Ce tri est intéressant parce qu'il est efficace sur des listes presque déjà triées. Sur une liste complètement aléatoire par contre, il devient lent.
Complexité : pire cas $O(n^2)$, meilleur cas $O(n)$ quand la liste est déjà triée.

```
Test tri par insertion :
[2, 23, 29, 40]
[40, 44, 46, 53]
[54, 64, 78, 85]
[86, 90, 94, 94]
```

4.4 Tri rapide (Quick Sort)

```python
def tri_rapide(table):
    nb_colonnes = len(table[0])
    lst = aplatir(table)
    lst_triee = _quick_sort(lst)
    return reconstruire(lst_triee, nb_colonnes)


def _quick_sort(lst):
    if len(lst) <= 1:
        return lst

    pivot = lst[len(lst) // 2]
    gauche = [x for x in lst if x < pivot]
    milieu  = [x for x in lst if x == pivot]
    droite  = [x for x in lst if x > pivot]

    return _quick_sort(gauche) + milieu + _quick_sort(droite)
```

Le tri rapide est un algorithme qui utilise le principe diviser pour régner. On choisit un élément appelé pivot, ici l'élément du milieu de la liste. On sépare ensuite tous les éléments en trois groupes : ceux qui sont plus petits que le pivot, ceux qui sont égaux, et ceux qui sont plus grands. On applique ensuite le même procédé récursivement sur chacun des groupes, jusqu'à ce que chaque groupe ne contienne plus qu'un seul élément.
La récursivité était la partie la plus difficile à comprendre au départ. La fonction **_quick_sort** s'appelle elle-même sur des portions de plus en plus petites de la liste, jusqu'à atteindre le cas de base qui est une liste d'un seul élément.
C'est l'algorithme le plus rapide en pratique parmi ceux qu'on a implémentés, surtout sur de grandes listes.
Complexité : cas moyen O(n log n), pire cas O(n^2) quand le pivot est toujours le plus grand ou le plus petit élément.

```
Test tri rapide :
[2, 23, 29, 40]
[40, 44, 46, 53]
[54, 64, 78, 85]
[86, 90, 94, 94]
```

4.5 Tri par fusion (Merge Sort)

```python
def tri_fusion(table):
    nb_colonnes = len(table[0])
    lst = aplatir(table)
    lst_triee = _merge_sort(lst)
    return reconstruire(lst_triee, nb_colonnes)


def _merge_sort(lst):
    if len(lst) <= 1:
        return lst

    milieu = len(lst) // 2
    gauche = _merge_sort(lst[:milieu])
    droite = _merge_sort(lst[milieu:])

    return _fusionner(gauche, droite)
```

```python
def _fusionner(gauche, droite):
    resultat = []
    i = j = 0

    while i < len(gauche) and j < len(droite):
        if gauche[i] <= droite[j]:
            resultat.append(gauche[i])
            i += 1
        else:
            resultat.append(droite[j])
            j += 1

    resultat.extend(gauche[i:])
    resultat.extend(droite[j:])

    return resultat
```

Le tri par fusion utilise aussi le principe diviser pour régner, comme le tri rapide. La différence c'est que ici on divise toujours la liste en deux moitiés égales, on trie chaque moitié récursivement, puis on fusionne les deux moitiés triées en une seule liste triée.
La fusion est l'opération clé. On compare le premier élément de chaque moitié, on prend le plus petit, et on avance dans cette moitié. On répète jusqu'à épuisement d'une des deux moitiés, puis on ajoute simplement le reste de l'autre.
Ce qui distingue ce tri des autres c'est sa stabilité : il donne toujours O(n log n) peu importe l'état de la liste au départ. C'est pour ca qu'on l'a utilisé pour trier le tableau avant les recherches binaire et dichotomique, parce qu'on sait qu'il sera toujours fiable.
Complexité : O(n log n) dans tous les cas, meilleur cas, cas moyen et pire cas.

```
Test tri par fusion :
[2, 23, 29, 40]
[40, 44, 46, 53]
[54, 64, 78, 85]
[86, 90, 94, 94]
```

5. Algorithmes de recherche

Les algorithmes de recherche permettent de trouver une valeur dans le tableau et de retourner sa position. Un point important : la recherche binaire et la recherche par dichotomie nécessitent obligatoirement un tableau trié pour fonctionner correctement. C'est pour ca qu'on applique le tri fusion avant de les utiliser.

5.1 Recherche linéaire

```python
def recherche_lineaire(table, cible):
    for i, ligne in enumerate(table):
        for j, element in enumerate(ligne):
            if element == cible:
                return (i, j)

    return (-1, -1)
```

La recherche linéaire est la plus simple. On parcourt le tableau case par case, ligne par ligne, jusqu'à trouver la valeur cherchée. Si on la trouve, on retourne sa position sous forme de tuple (ligne, colonne). Si on arrive à la fin sans la trouver, on retourne (-1, -1).
Son avantage principal est qu'elle fonctionne sur un tableau non trié. Son inconvénient c'est que dans le pire cas on parcourt tout le tableau sans rien trouver.
Complexité : pire cas O(n), meilleur cas O(1) quand la valeur est en première position.

```
Valeur recherchee (existe)      : 54
Valeur recherchee (n'existe pas): 9999
```

```
Table originale :
[2, 29, 64, 54]
[94, 23, 44, 40]
[90, 40, 86, 94]
[53, 85, 46, 78]
```

```
Test recherche lineaire (valeur existante) :
   SUCCES - Trouve a (ligne=0, colonne=3)

Test recherche lineaire (valeur absente) :
   SUCCES - Valeur absente correctement non trouvee
```

On cherche la valeur 54, la recherche linéaire la trouve dans le tableau original à la ligne 0, colonne 3.

5.2 Recherche binaire

```python
def recherche_binaire(table, cible):
    lst = aplatir(table)
    gauche = 0
    droite = len(lst) - 1

    while gauche <= droite:
        milieu = (gauche + droite) // 2

        if lst[milieu] == cible:
            return milieu
        elif lst[milieu] < cible:
            gauche = milieu + 1
        else:
            droite = milieu - 1

    return -1
```

La recherche binaire est beaucoup plus efficace que la recherche linéaire, mais elle nécessite obligatoirement un tableau trié. On aplatit d'abord le tableau en liste 1D, puis on définit deux bornes : gauche au début et droite à la fin. A chaque étape on calcule le milieu avec la formule **milieu = (gauche + droite) // 2**. Si l'élément au milieu est égal à la valeur cherchée, on retourne son index. S'il est plus petit, on déplace la borne gauche vers la droite. S'il est plus grand, on déplace la borne droite vers la gauche. On répète jusqu'à trouver ou jusqu'à ce que les bornes se croisent.
L'avantage de cette méthode c'est qu'à chaque étape on élimine la moitié des éléments restants. Sur une liste de 1000 éléments par exemple, on trouve la valeur en 10 étapes maximum au lieu de 1000.
La fonction retourne l'index de la valeur dans la liste aplatie, et -1 si elle n'est pas trouvée.
Complexité : O(log n) dans tous les cas.

```
Valeur recherchee (existe)      : 54
Valeur recherchee (n'existe pas): 9999

Table triee utilisee pour recherche binaire et dichotomie :
[2, 23, 29, 40]
[40, 44, 46, 53]
[54, 64, 78, 85]
[86, 90, 94, 94]
```

```
Test recherche binaire (valeur existante) :
   SUCCES - Trouve a l'index 8 dans la liste aplatie

Test recherche binaire (valeur absente) :
   SUCCES - Valeur absente correctement non trouvee
```

5.3 Recherche par dichotomie

La recherche par dichotomie fonctionne sur le même principe que la recherche binaire. La différence principale est qu'elle est implémentée de façon récursive : la fonction **_dichotomie** s'appelle elle-même avec des bornes réduites à chaque fois, jusqu'à trouver la valeur ou jusqu'à ce que les bornes se croisent.
Le mot dichotomie signifie couper en deux. A chaque appel récursif, on coupe l'espace de recherche en deux, ce qui explique sa rapidité.
Comme la recherche binaire, elle retourne l'index de la valeur dans la liste aplatie et -1 si elle n'est pas trouvée. Les deux donnent d'ailleurs le même résultat dans nos tests, ce qui est normal puisqu'elles appliquent le même principe, l'une de façon itérative et l'autre de façon récursive.
Complexité : O(log n) dans tous les cas.

```
Test recherche par dichotomie (valeur existante) :
   SUCCES - Trouve a l'index 8 dans la liste aplatie

Test recherche par dichotomie (valeur absente) :
   SUCCES - Valeur absente correctement non trouvee
```

6. Tests et validation

Pour valider le module, nous avons créé un fichier de test séparé **test_tableau2d.py**. Ce fichier génère un tableau aléatoire et teste chaque fonction une par une.
Pour les tris, on a vérifié que tous les algorithmes donnent exactement le même résultat en comparant leurs sorties. Si tous sont identiques, le test affiche SUCCES. C'est une façon simple de détecter une erreur dans un algorithme.

Pour les recherches, on a testé deux cas : une valeur qui existe dans le tableau, et une valeur absente (9999). Dans les deux cas les trois algorithmes répondent correctement, ce qui confirme que le module fonctionne comme attendu.

```python
import tableau2d

print("=" * 60)
print("   TESTS DU MODULE tableau2d.py")
print("=" * 60)

# ------------------------------------------------------------
# GENERATION DE LA TABLE DE TEST
# ------------------------------------------------------------

print("\nGeneration de la table 2D aleatoire...")
table = tableau2d.generer_table(4, 4, 1, 99)
print("Table originale :")
tableau2d.afficher_table(table)
print("Liste aplatie :", tableau2d.aplatir(table))
```

```python
# ------------------------------------------------------------
# TESTS DES TRIS
# ------------------------------------------------------------
print("\n" + "=" * 60)
print("   TESTS DES TRIS")
print("=" * 60)

print("\nTest tri a bulle :")
result = tableau2d.tri_bulle(table)
tableau2d.afficher_table(result)

print("\nTest tri par selection :")
result = tableau2d.tri_selection(table)
tableau2d.afficher_table(result)

print("\nTest tri rapide :")
result = tableau2d.tri_rapide(table)
tableau2d.afficher_table(result)

print("\nTest tri par insertion :")
result = tableau2d.tri_insertion(table)
tableau2d.afficher_table(result)

print("\nTest tri par fusion :")
result = tableau2d.tri_fusion(table)
tableau2d.afficher_table(result)
```

```python
print("\nVerification - tous les tris donnent le meme resultat :")
bulle     = tableau2d.aplatir(tableau2d.tri_bulle(table))
selection = tableau2d.aplatir(tableau2d.tri_selection(table))
rapide    = tableau2d.aplatir(tableau2d.tri_rapide(table))
insertion = tableau2d.aplatir(tableau2d.tri_insertion(table))
fusion    = tableau2d.aplatir(tableau2d.tri_fusion(table))

if bulle == selection == rapide == insertion == fusion:
    print("   SUCCES - Tous les tris produisent le meme resultat !")
else:
    print("   ERREUR - Les tris ne donnent pas le meme resultat !")
```

```python
# ------------------------------------------------------------
# TESTS DES RECHERCHES
# ------------------------------------------------------------

print("\n" + "=" * 60)
print("   TESTS DES RECHERCHES")
print("=" * 60)

valeur_existante = tableau2d.aplatir(table)[3]
valeur_absente = 9999

print(f"\nValeur recherchee (existe)      : {valeur_existante}")
print(f"Valeur recherchee (n'existe pas): {valeur_absente}")

table_triee = tableau2d.tri_fusion(table)
print("\nTable triee utilisee pour recherche binaire et dichotomie :")
tableau2d.afficher_table(table_triee)

print("\nTest recherche lineaire (valeur existante) :")
pos = tableau2d.recherche_lineaire(table, valeur_existante)
if pos != (-1, -1):
    print(f"   SUCCES - Trouve a (ligne={pos[0]}, colonne={pos[1]})")
else:
    print("   ERREUR - Valeur non trouvee alors qu'elle existe !")

print("\nTest recherche lineaire (valeur absente) :")
pos = tableau2d.recherche_lineaire(table, valeur_absente)
if pos == (-1, -1):
    print("   SUCCES - Valeur absente correctement non trouvee")
else:
    print("   ERREUR - Valeur trouvee alors qu'elle n'existe pas !")
```

```python
print("\nTest recherche binaire (valeur existante) :")
idx = tableau2d.recherche_binaire(table_triee, valeur_existante)
if idx != -1:
    print(f"   SUCCES - Trouve a l'index {idx} dans la liste aplatie")
else:
    print("   ERREUR - Valeur non trouvee alors qu'elle existe !")

print("\nTest recherche binaire (valeur absente) :")
idx = tableau2d.recherche_binaire(table_triee, valeur_absente)
if idx == -1:
    print("   SUCCES - Valeur absente correctement non trouvee")
else:
    print("   ERREUR - Valeur trouvee alors qu'elle n'existe pas !")

print("\nTest recherche par dichotomie (valeur existante) :")
idx = tableau2d.recherche_dichotomie(table_triee, valeur_existante)
if idx != -1:
    print(f"   SUCCES - Trouve a l'index {idx} dans la liste aplatie")
else:
    print("   ERREUR - Valeur non trouvee alors qu'elle existe !")

print("\nTest recherche par dichotomie (valeur absente) :")
idx = tableau2d.recherche_dichotomie(table_triee, valeur_absente)
if idx == -1:
    print("   SUCCES - Valeur absente correctement non trouvee")
else:
    print("   ERREUR - Valeur trouvee alors qu'elle n'existe pas !")
```

7. Difficultés rencontrées

La première difficulté était de comprendre pourquoi on devait passer par une liste 1D pour trier. Au début ce n'était pas évident de voir pourquoi on ne pouvait pas trier directement le tableau 2D.

La récursivité dans le tri rapide et le tri par fusion était aussi difficile à suivre au début, parce que la fonction s'appelle elle-même et il faut bien visualiser ce qui se passe à chaque étape.

Enfin, on a remarqué que la recherche linéaire et les deux autres ne cherchent pas dans le même tableau, ce qui explique pourquoi elles ne retournent pas la position de la même façon.

Conclusion

Ce projet nous a permis de comprendre concrètement le fonctionnement des algorithmes de tri et de recherche, en les implémentant nous-mêmes sur une structure de données à deux dimensions.

On a pu constater que tous les algorithmes de tri donnent le même résultat final, mais leurs performances sont très différentes. Le tri à bulles et le tri par sélection sont simples à comprendre mais deviennent lents sur de grandes listes. Le tri rapide et le tri par fusion sont plus complexes à implémenter, surtout avec la récursivité, mais ils sont bien plus efficaces.

Pour les recherches, la recherche linéaire est la plus simple et fonctionne sur n'importe quel tableau. La recherche binaire et la dichotomie sont plus rapides mais nécessitent un tableau trié.

Voici un tableau récapitulatif des complexités :

Algorithme	Meilleur cas	Pire cas
Tri à bulles	$O(n)$	$O(n^2)$
Tri par sélection	$O(n^2)$	$O(n^2)$
Tri par insertion	$O(n)$	$O(n^2)$
Tri rapide	$O(n \log n)$	$O(n^2)$
Tri par fusion	$O(n \log n)$	$O(n \log n)$
Recherche linéaire	$O(1)$	$O(n)$
Recherche binaire	$O(1)$	$O(\log n)$
Recherche dichotomie	$O(1)$	$O(\log n)$

Ce projet nous a aussi appris l'importance de bien tester son code. Le fichier de test nous a permis de détecter rapidement si quelque chose ne fonctionnait pas.